

Quiz #3 Answer Sheet

CS 225 Data Structures

Spring 2026

7:00-8:00pm, Thursday, March 26

Name	Hanbin Zheng
ZJU ID	
Lab TA	

- This is a **closed book** and **closed notes** exam. No electronic aids are allowed, either.
- You should have 4 problems total on 5 pages. You must write on the exam.
- Unless otherwise stated in a problem, assume the best possible design of a particular implementation is being used.
- Unless the problem specifically says otherwise, (1) assume the code compiles, and thus any compiler error is an exam typo (though hopefully there are not any typos), and (2) assume you are NOT allowed to write any helper methods to help solve the problem, nor are you allowed to use additional arrays, lists, or other collection data structures unless we have said you can.
- Please put your name at the top of each page.
- Please fill in your **Lab Section** or your **Lab TA's name** to help us organize and grade your papers.
- Don't panic and good luck!

Problem 1 8/10

Problem 2 9.5/10

Problem 3 9/10

Problem 4 4.5/10

Total 31/40



Problem 1 (10 points)

1.

(1) d (2) c (3) ab

2. Array content after step 3:

["c", "s", "2", "2", "5"]

Array content after step 7:

["c", "s", "2", "5", "2"]**Problem 2 (10 points): Array List and Linked List Operations**

1.

(a) arr.size - 1(b) arr.size + arr.capacity + constant2. amortized time for resize: $O(1)$ $\rightarrow O\left(\frac{1+2+2^2+\dots+2^{k-1}}{c \cdot 2^k}\right) = O\left(1 + \frac{c}{2^k}\right) = O(1)$ insertion: arr.sizeThus, $O(arr.size) + O(1) \rightarrow O(arr.size)$

3.

(a) arr.size - 1; idx (b) arr.size

4.

unsorted array list: $O(n)$; sorted array list (using best method): $O(\log n)$ 5. No, still $O(n)$

We need first find the element and then insert it.

For finding the element, we reduce it from n to $\log_2 n$.But we still need to insert, which is also n operationsThus, still $O(n)$

6.

(a) list.head; n (b) $O(1)$

7.

unsorted array list: $O(n)$; sorted array list: $O(n)$

8.

Not correct. For "finding" operation, we need $\log_2 n$ times of finding. But we need $O(n)$ times of moving during each "finding", which makes it slower than $O(\log_2 n)$.



Problem 3 (10 points): Stack and Queue implementation

1.

template <class T>

void Stack<T>::push(const T& x) {

if (count_ == size_) { enlarge(); }

arr_[count_++] = x;

}

2.

template <class T>

T Stack<T>::pop() {

if (isEmpty()) { exit(-1); }

T tmp = arr_[count_ - 1];

count_ --;

}

慢通怎么没返回值

3.

template <class T>

void Queue<T>::enqueue(const T& x) {

if (isFull()) { enlarge(); }

arr_[back_] = x;

back_ = (back_ + 1) % size_;

count_ ++;

}



4.

template <class T>

T Queue<T>::dequeue() {

```

    if (isEmpty()) {exit(-1);}
    T tmp = arr[front-1];
    front = (front + 1) % size;
    count--;

```

-0.5

递增怎么没返回值

}

5.

template <class T>

bool Stack<T>::isEmpty() const {

```

    return (count == 0);

```

}

Problem 4 (10 points): iterator & recursion

1.

(1) 5 times

(2)

20

(3)

~~0~~ c

-2



(4)

int mystery_iter(vector<int>& v) {

-1.5

int value = 0;

for (auto it : v) {

value = value + *it;

}

return value;

}

2.

(1)

times. 傻逼吧, ~~读题读全了~~.

(2)

 ~~$\log_2(n)$~~ $n=16$, 用 $\lceil \log_2 n \rceil$ 或者 5

(3)

d.

